The market can respond to a command by accepting the changes or not. It can also respond with its current status when asked:

```
object Market {
  ...

  sealed trait Response extends CborSerializable
  final case object Accepted extends Response
  final case class CurrentState(status: Status) extends Response
  final case class RequestUnaccepted(reason: String) extends Response
  ...
}
```

Now the state. The market starts with `UninitializedState`. The state can then be updated, canceled, or closed. This `State` and the rest have a `Status` that holds all the market variables. These are the different states:

```
object Market {
  ...

  sealed trait State extends CborSerializable {
    def status: Status;                                  ❶
  }
  final case class UninitializedState (status: Status) extends State
  final case class OpenState(status: Status) extends State
  final case class ClosedState(status: Status) extends State
  final case class CancelledState(status: Status) extends State
  ...
}
```

❶ Must be implemented in any class extending this trait

`Status` has the market ID, the fixture, and the odds. And for convenience, there is a constructor to create an empty market:

```
object Market {
  ...

  final case class Status(marketId: String, fixture: Fixture, odds: Odds)
     extends CborSerializable
```